



MODUL 10 · VALIDASI DATA & FORMS

react-hook-form *+ Zod Resolver*



Client-side real-time validation dengan shadcn/ui. Satu schema untuk client dan server.

Setup react-hook-form + Zod

bun add react-hook-form @hookform/resolvers. zodResolver menghubungkan Zod ke react-hook-form.

```
# bun add react-hook-form @hookform/resolvers zod

"use client"
import { useForm } from "react-hook-form"
import { zodResolver } from "@hookform/resolvers/zod"
import { z } from "zod"

const schema = z.object({
  title: z.string().min(3, "Min 3 karakter"),
  content: z.string().min(10, "Terlalu pendek"),
})
type FormValues = z.infer<typeof schema>

function PostForm() {
  const form = useForm<FormValues>({
    resolver: zodResolver(schema),
    defaultValues: { title: "", content: "" },
  })

  function onSubmit(data: FormValues) {
    // data sudah typed dan validated!
    console.log(data)
  }

  return (
    <form onSubmit={form.handleSubmit(onSubmit)}>
      <input {...form.register("title")} />
      {form.formState.errors.title && (
        <p>{form.formState.errors.title.message}</p>
      )}
      <button type="submit">Simpan</button>
    </form>
  )
}
```

useForm returns

form.register(name):
Props untuk native input

form.formState.errors:
Errors per field dari Zod

form.handleSubmit(fn):
Jalankan fn jika validasi pass

form.watch(field):
Live value field

form.setValue(f, v):
Set value programatically

form.reset(values?):
Reset ke default

form.setError(f, err):
Set error dari server

form.formState.isSubmitting:
true saat submit berjalan

Z O D & F O R M S

shadcn/ui Form Components

FormField + FormItem + FormMessage = accessible form dengan error display otomatis.

```
"use client"
import { useForm } from "react-hook-form"
import { zodResolver } from "@hookform/resolvers/zod"
import {
  Form, FormControl, FormField, FormItem,
  FormLabel, FormMessage,
} from "@components/ui/form"
import { Input } from "@components/ui/input"
import { Textarea } from "@components/ui/textarea"
import { Button } from "@components/ui/button"
import { createPostSchema, type CreatePostInput }
  from "@lib/validations/post"

export function CreatePostForm() {
  const form = useForm<CreatePostInput>({
    resolver: zodResolver(createPostSchema),
    defaultValues: { title:"", content:"", published:false },
  })

  async function onSubmit(data: CreatePostInput) {
    const result = await createPost(/* ... */)
    if (!result.success) {
      // Map server errors ke form fields
      Object.entries(result.errors).forEach(([field, msgs]) =>
        form.setError(field as keyof CreatePostInput,
          { message:msgs[0] })
      )
    }
  }

  return (
    <Form {...form}>
      <form onSubmit={form.handleSubmit(onSubmit)} className="space-y-4">
        <FormField control={form.control} name="title"
          render={({ field }) => (
            <FormItem>
              <FormLabel>Judul Post</FormLabel>
              <FormControl>
                <Input placeholder="Judul..." {...field} />
              </FormControl>
              <FormMessage /> { /* error otomatis dari Zod! */}
            </FormItem>
          )} />
        <Button type="submit" disabled={form.formState.isSubmitting}>
          {form.formState.isSubmitting ? "Menyimpan..." : "Simpan"}
        </Button>
      </form>
    </Form>
  )
}
```

Evaluasi Pemahaman

Jawab sebelum lanjut.

Q1 — Apa fungsi zodResolver dalam react-hook-form setup?

- A) Menggantikan react-hook-form B) Menghubungkan Zod schema ke react-hook-form sebagai validation engine C) Mengkonversi FormData ke object

Q2 — Bagaimana menampilkan server error di field 'slug' menggunakan react-hook-form?

- A) setError('slug', 'pesan') B) form.setError('slug', { message: 'Slug sudah dipakai' }) C) errors.slug = 'pesan'

Q3 — Kapan onSubmit di react-hook-form dijalankan?

- A) Selalu saat form submit B) Hanya jika SEMUA validasi Zod lolos — data sudah validated saat masuk ke onSubmit C) Hanya untuk form yang tidak punya validasi

Tugas Mandiri

Buat form lengkap dengan react-hook-form + Zod + shadcn/ui.

01 Install Setup

`bun add react-hook-form @hookform/resolvers`. Buat `CreatePostForm` dengan `zodResolver(createPostSchema)`.

02 Form Fields

Buat form: title (Input), content (Textarea), published (Switch). Setiap field punya `FormLabel`, `FormControl`, `FormMessage`.

03 Server Integration

Hubungkan ke `createPost` Server Action. Tampilkan server errors dengan `form.setError()`. Button disabled saat `isSubmitting`.

04 Success Flow

Setelah sukses: `form.reset()`, toast notification, redirect ke `/blog/[slug]`. Form bisa dipakai ulang setelah submit.



`bunx shadcn@latest add form` — install shadcn Form components yang wrap react-hook-form.

Yang Sudah Kita Pelajari

- ✓ zodResolver: bridge Zod → react-hook-form. Satu schema untuk client validation.
- ✓ shadcn Form + FormField + FormMessage: accessible form dengan auto error display.
- ✓ form.setError(field, {message}): tampilkan server errors ke form fields.
- ✓ Two-layer validation: client (real-time UX) + server (security & business rules).
- ✓ form.formState.isSubmitting: disable button saat submit untuk prevent double-submit.

→ Selanjutnya: **Bab 5 — React 19 Form Hooks: useActionState & useFormStatus**

